

SOC Engineering Lab Portfolio & Advanced Diagnostic Manual

1.Phase 1: Architecture Blueprint & Network Topology Design

Before powering on any guest operating systems, the isolated layer-2 switch environment and target IP layout must be mapped out to prevent sub-network collisions.

A.Host-Only Network Adapter Architecture (VirtualBox Core)

Isolation from the host's primary gateway is maintained via a secondary virtualization switch fabric.

Interface Profile Name	IPv4 Prefix Address	DHCP Daemon Status	Assigned Node Bounds
VirtualBox Host-Only Ethernet Adapter	192.168.56.1/24	Enabled	Default Local Routing Link
VirtualBox Host-Only Ethernet Adapter #2	192.168.30.1/24	Enabled (Server: 192.168.30.2)	Pool Bounds: 192.168.30.3 - 192.168.30.254

B.Dynamic Network Asset Mapping Matrix

Virtual Asset Node	Virtual NIC Device	Expected Dynamic IP	Operational Role / Function
Host Gateway Interface	Host-Only Adapter #2	192.168.30.1	Virtual Gateway interface on the host side.
VirtualBox Engine DHCP	DHCP Daemon Server	192.168.30.2	The virtual DHCP daemon handing out addresses.
Win10 Target Node	Ethernet adapter	192.168.30.3	RDP Target Node / First leased IP from DHCP pool.
SOC-SIEM Core Server	enp0s3 (Ubuntu)	192.168.30.4	Splunk Core Engine / Second leased IP from DHCP pool.
Ubuntu SSH Target Node	enp0s3 (Ubuntu)	192.168.30.5	SSH Target Node / Third leased IP from DHCP pool.
Arch Linux Attacker	enp0s3 (Arch)	192.168.30.6	Penetration Suite / Fourth leased IP from DHCP pool.

2. Phase 2: Host Provisioning & Tool Acquisition Sequence

With the network fabric established, individual endpoints are configured with defensive monitoring assets and offensive testing toolsets.

A.Arch Linux Attack Suite Setup & Validation

Step 1: Update system packages and install Nmap and THC-Hydra simultaneously via package manager:

```
sudo pacman -Syu nmap hydra --noconfirm
```

 **Screenshot Reference: [nmap - hydra.png]**

Verifies running package database updates for core and extra repositories, and downloading the utilities cleanly.

```
[khan@archlinux ~]$ sudo pacman -Syu nmap hydra --noconfirm
[sudo] password for khan:
:: Synchronizing package databases...
   core                               126.6 KiB   118 KiB/s  00:01 [#####] 100%
  extra                               8.3 MiB   1113 KiB/s  00:08 [#####] 100%
```

Step 2: Verify correct binary installations and software revisions:

```
nmap --version
hydra -h | head -n 1
```

 **Screenshot Reference: [nmap.png] and [hydra.png]**

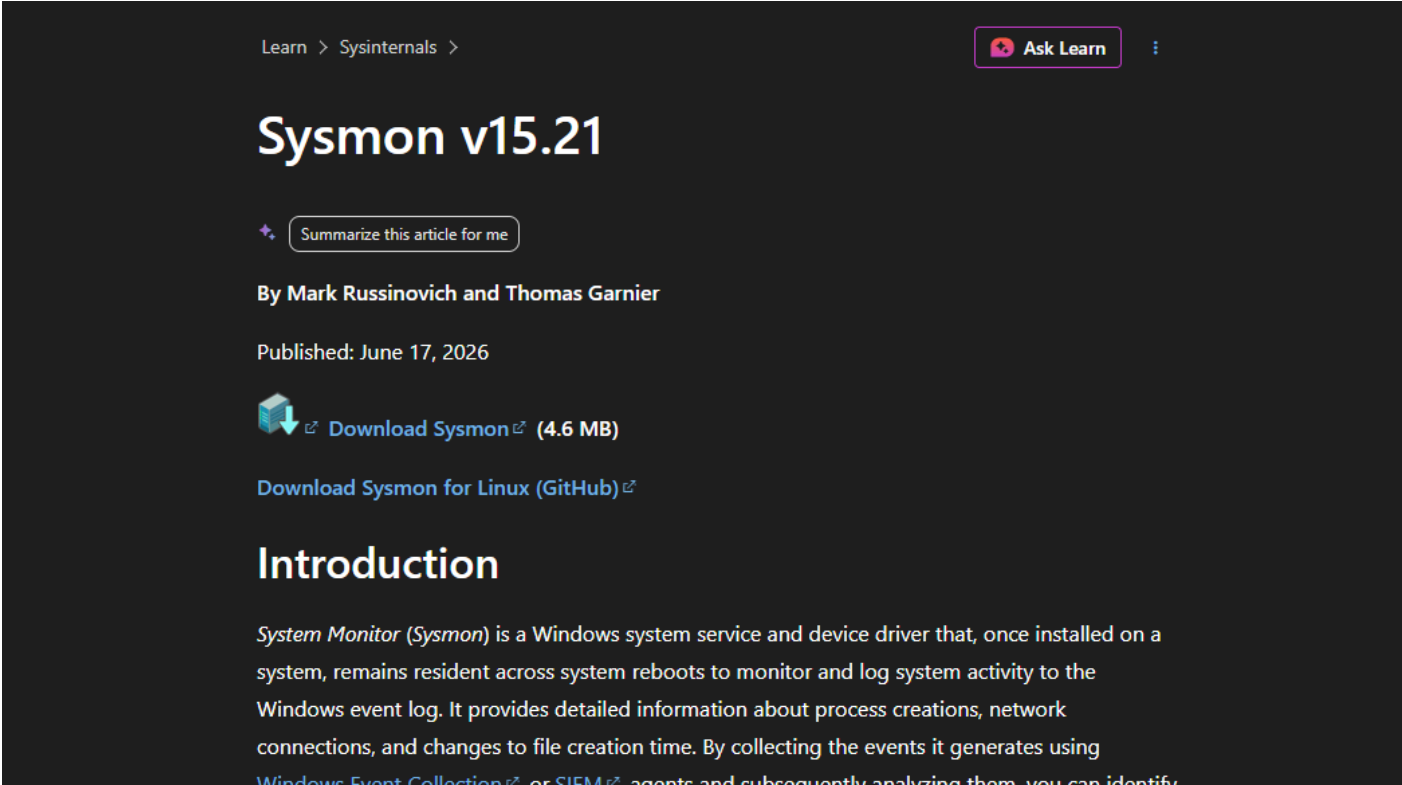
Confirms Nmap v7.98 platform alignment and Hydra v9.6 framework release are running on the system architecture.

```
[khan@archlinux ~]$ nmap --version
Nmap version 7.98 ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
```

B.Windows 10 Target Proactive Host Hardening (Sysmon Lifecycle Deployment)

Step 1: Download Microsoft System Monitor (Sysmon v15.21) inside the virtualized Windows environment:

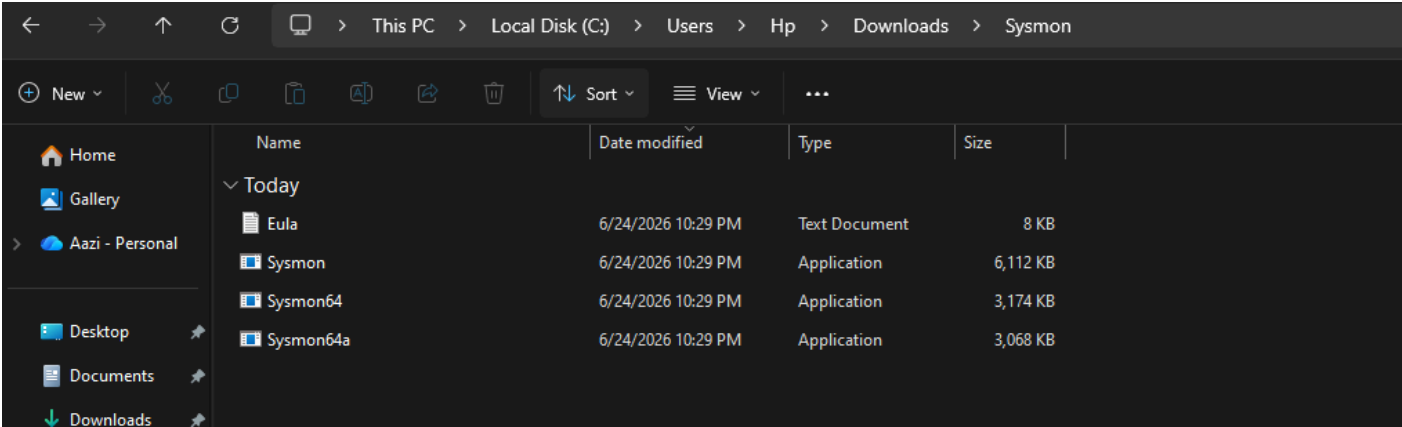
 **Screenshot Reference: [sysmon download.png]**
Displays the official Sysinternals release portal download page for the tool.

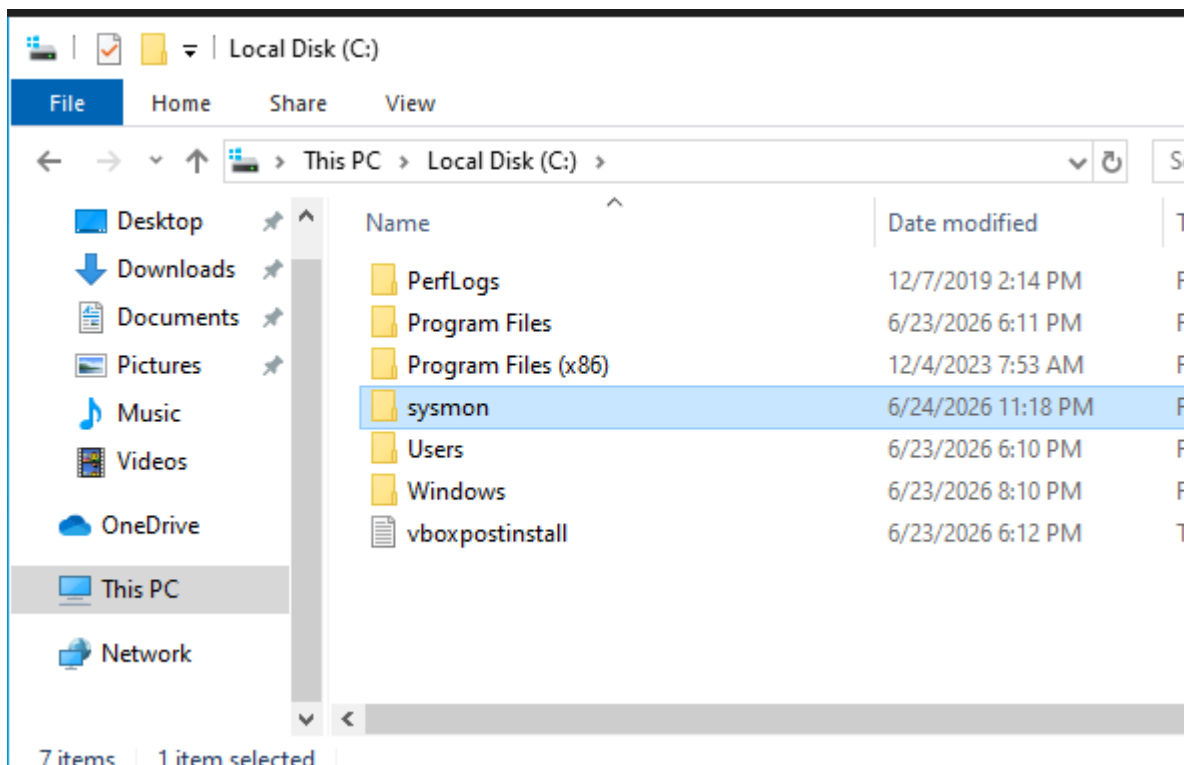


Step 2: Unzip the downloaded archive and structure the executables inside a root filesystem directory:

```
Path structured: C:\sysmon\
```

 **Screenshot Reference: [sysmon.png] and [sysmon transfered.png]**
Confirms file transfers, unpacking the tool, and organizing files like Eula, Sysmon, and Sysmon64.exe inside the C:\sysmon folder.





Step 3: Open an elevated Administrator command prompt context and initialize the driver using a dedicated configuration template file:

```
cd C:\sysmon
sysmon64.exe -accepteula -i sysmonconfig.xml
```

Screenshot Reference: [sys_end.png]

Confirms driver initialization, showing Sysmon v15.21 active, schema v4.50 loaded, and schema v4.91 established on the host.

3. Phase 3: Local Network Diagnostics, Firewalls, and Dynamic Discovery

Once all nodes were online, serious connectivity issues appeared across the virtual switch fabric, requiring manual low-level diagnosis before proceeding.

A. The Connectivity Barricade (ICMP Loss & Mismapped Routes)

```
[khan@archlinux ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:66:4f:57 brd ff:ff:ff:ff:ff:ff
    altname enx080027664f57
    inet 192.168.30.6/24 brd 192.168.30.255 scope global dynamic noprefixroute enp0s3
        valid_lft 510sec preferred_lft 510sec
    inet6 fe80::59a1:ee74:5416:eaef/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:86:d8:aa brd ff:ff:ff:ff:ff:ff
    altname enx08002786d8aa
    inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic noprefixroute enp0s8
        valid_lft 86310sec preferred_lft 86310sec
    inet6 fd17:625c:f037:3:a1ad:d248:9dff:d44f/64 scope global dynamic noprefixroute
        valid_lft 86383sec preferred_lft 14383sec
    inet6 fe80::5aa8:f91b:1108:4c92/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 12:7b:b8:0f:78:37 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
[khan@archlinux ~]$ ip route
default via 10.0.3.2 dev enp0s8 proto dhcp src 10.0.3.15 metric 101
10.0.3.0/24 dev enp0s8 proto kernel scope link src 10.0.3.15 metric 101
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.30.0/24 dev enp0s3 proto kernel scope link src 192.168.30.6 metric 100
[khan@archlinux ~]$ ping 192.168.30.3
PING 192.168.30.3 (192.168.30.3) 56(84) bytes of data.
From 192.168.30.6 icmp_seq=1 Destination Host Unreachable
ping: sendmsg: No route to host
From 192.168.30.6 icmp_seq=2 Destination Host Unreachable
From 192.168.30.6 icmp_seq=3 Destination Host Unreachable
From 192.168.30.6 icmp_seq=5 Destination Host Unreachable
From 192.168.30.6 icmp_seq=6 Destination Host Unreachable
From 192.168.30.6 icmp_seq=7 Destination Host Unreachable
^C
--- 192.168.30.3 ping statistics ---
 9 packets transmitted, 0 received, +6 errors, 100% packet loss, time 8219ms
pipe 3
[khan@archlinux ~]$
```

```
vboxuser@SOC-SIEM:~$ ping 192.168.30.5
PING 192.168.30.5 (192.168.30.5) 56(84) bytes of data.
From 192.168.30.4 icmp_seq=1 Destination Host Unreachable
From 192.168.30.4 icmp_seq=2 Destination Host Unreachable
From 192.168.30.4 icmp_seq=3 Destination Host Unreachable
From 192.168.30.4 icmp_seq=4 Destination Host Unreachable
From 192.168.30.4 icmp_seq=5 Destination Host Unreachable
From 192.168.30.4 icmp_seq=6 Destination Host Unreachable
From 192.168.30.4 icmp_seq=7 Destination Host Unreachable
From 192.168.30.4 icmp_seq=8 Destination Host Unreachable
From 192.168.30.4 icmp_seq=9 Destination Host Unreachable
From 192.168.30.4 icmp_seq=10 Destination Host Unreachable
From 192.168.30.4 icmp_seq=11 Destination Host Unreachable
From 192.168.30.4 icmp_seq=12 Destination Host Unreachable
From 192.168.30.4 icmp_seq=13 Destination Host Unreachable
From 192.168.30.4 icmp_seq=14 Destination Host Unreachable
From 192.168.30.4 icmp_seq=15 Destination Host Unreachable

--- 192.168.30.5 ping statistics ---
16 packets transmitted, 0 received, +15 errors, 100% packet loss, time 15352ms
pipe 4
^Cvboxuser@SOC-SIEM:~$
```

Error Symptoms & Diagnostic Logs:

From the Arch Linux terminal interface (`192.168.30.6`), issuing an ICMP ping command to the Windows Target (`192.168.30.3`) threw immediate errors:

```
From 192.168.30.6 icmp_seq=1 Destination Host Unreachable
ping: sendmsg: No route to host
```

 **Screenshot Reference: [image_5a7880.png]** (Shows 9 packets transmitted, 0 received, 100% packet loss).

Similarly, attempting to trace a link from the SOC-SIEM Server (`192.168.30.4`) to the secondary Ubuntu target node (`192.168.30.5`) yielded a total communication freeze:

```
From 192.168.30.4 icmp_seq=1 Destination Host Unreachable
```

 **Screenshot Reference: [image_5a78be.png]** (Shows 16 packets transmitted, 0 received, 100% packet loss).

B.Root Cause Isolation & Structural Adjustments

The connectivity failures stemmed from stale ARP neighbor tables within the virtual switch interfaces and strict default inbound firewall rules on the Windows target.

Step 1: Flush stale neighborhood routing entries to force new hardware address lookups:

```
sudo ip neigh flush all
```

Step 2: Verify active hosts using local neighborhood queries and Nmap ARP ping checks:

```
ip neighbor show
sudo nmap -sn -PR 192.168.30.0/24
```

Step 3: Modify the Windows 10 target host-level firewall via PowerShell to explicitly permit local ICMP diagnostic traffic:

```
New-NetFirewallRule -DisplayName "Allow Lab Pings" -Direction Inbound -Action Allow -Protocol ICMPv4
```

```
Name                : {3adb7748-5c9f-4250-829e-74dd10b5d70a}
DisplayName          : Allow Lab Pings
Description          :
DisplayGroup         :
Group                :
Enabled              : True
Profile              : Any
Platform             : {}
Direction            : Inbound
Action               : Allow
EdgeTraversalPolicy  : Block
LooseSourceMapping   : False
LocalOnlyMapping     : False
Owner                :
PrimaryStatus        : OK
Status               : The rule was parsed successfully from the store. (65536)
EnforcementStatus    : NotApplicable
PolicyStoreSource    : PersistentStore
PolicyStoreSourceType : Local
```

4. Phase 4: Splunk Network Receiver Infrastructure Setup

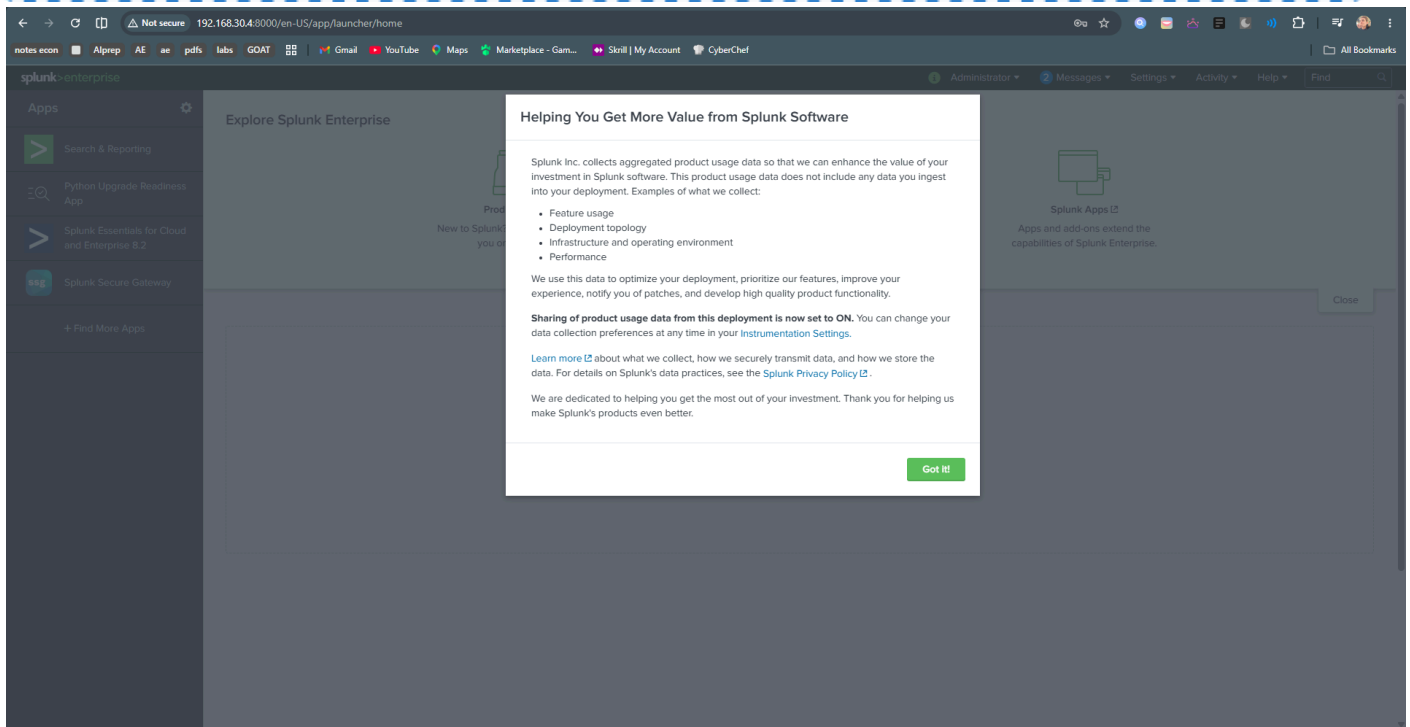
With reliable network connectivity verified across the subnet, the central ingestion engine was opened to listen for endpoint forwarder telemetry.

Step 1: Access the central SIEM node over the local host-only subnet interface browser:

URL: `http://192.168.30.4:8000`

Screenshot Reference: [splunk.png]

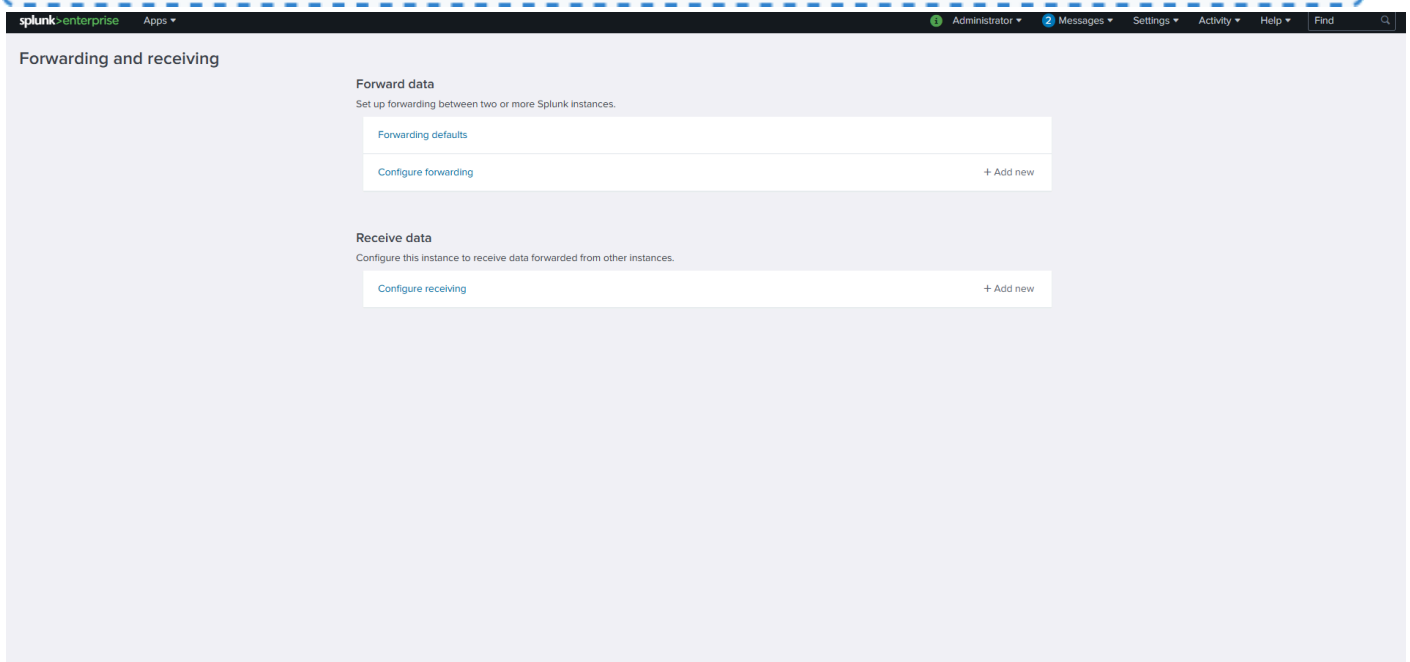
Shows successful initial web UI portal response and configuration screens over the 192.168.30.4 loop.



Step 2: Navigate to Settings > Forwarding and receiving inside the Splunk dashboard interface:

Screenshot Reference: [splunkforwardingaddnew.png]

Shows navigating to the management page to bind data ingress routing channels.



Step 3: Under 'Receive Data', select 'Configure receiving > Add new', and define the TCP ingestion port parameter:

```
Listen on this port: 9997
```



Screenshot Reference: [splunkreceivinghost.png]

Confirms setting up the TCP data link on port 9997 to receive logs from forwarder agents.

C.Phase 4-B: Dictionary Weaponization & Linux SSH Credential Stuffing

Step 1: Generate a customized local password wordlist utilizing a console terminal editor:

```
nano passwords.txt
```

Step 2: Populate the dictionary array structure directly with target variants:

```
passwordl23
adminl23
security2026
cybersecurity
vboxuser
```

Step 3: Verify wordlist size and line entry count strings via terminal diagnostics:

```
cat passwords.txt
wc -l passwords.txt
```

Step 4: Launch a high-velocity dictionary execution trail using THC-Hydra targeting the Ubuntu node to trigger authentication failures:

```
hydra -l vboxuser -P passwords.txt ssh://192.168.30.5 -t 4 -V
```

Execution Profile & Adversarial Impact:

This attack directly launched automated login attempts using the specified username and custom password list, successfully generating the 17 sequential **failed password** telemetry sequences that were picked up by the central SIEM.

D.Phase 4-C: Automated Windows RDP Credential Stuffing

Step 1: Execute an automated network brute-force execution path against the Windows RDP daemon interface using the engineered wordlist:

```
hydra -l LabAdmin -P passwords.txt rdp://192.168.30.3 -t 1 -V
```

This simulation populated the Windows Security Event Log with explicit Event ID 4625 (An account failed to log on) sequences, tracked in detail by the resident Sysmon process manager for threat inspection.

1.Phase 5: Splunk Network Receiver Infrastructure Setup

With reliable network connectivity verified across the subnet and attacks verified, the central ingestion engine was opened to listen for endpoint forwarder telemetry.

Step 1: Access the central SIEM node over the local host-only subnet interface browser:

```
URL: http://192.168.30.4:8000
```

Step 2: Navigate to Settings > Forwarding and receiving inside the Splunk dashboard interface:

Step 3: Under 'Receive Data', select 'Configure receiving > Add new', and define the TCP ingestion port parameter:

The screenshot shows the Splunk Enterprise dashboard interface. At the top, there is a navigation bar with the Splunk logo, 'enterprise' text, and a dropdown menu for 'Apps'. On the right side of the navigation bar, there are several tabs: 'Administrator', 'Messages', 'Settings', 'Activity', 'Help', and a search bar labeled 'Find'. Below the navigation bar, the main content area is titled 'Add new' with a breadcrumb trail: 'Forwarding and receiving > Receive data > Add new'. In the center of the main content area, there is a white modal window titled 'Configure receiving'. Inside this modal, the text 'Set up this Splunk instance to receive data from forwarder(s).' is displayed. Below this text, there is a label 'Listen on this port *' followed by a text input field containing the value '9997'. A small note below the input field states: 'For example, 9997 will receive data on TCP port 9997.' At the bottom right of the modal, there are two buttons: 'Cancel' and 'Save'.

```
Listen on this port: 9997
```

Screenshot Reference: [splunkreceivinghost.png]

Confirms setting up the TCP data link on port 9997 to receive logs from forwarder agents.

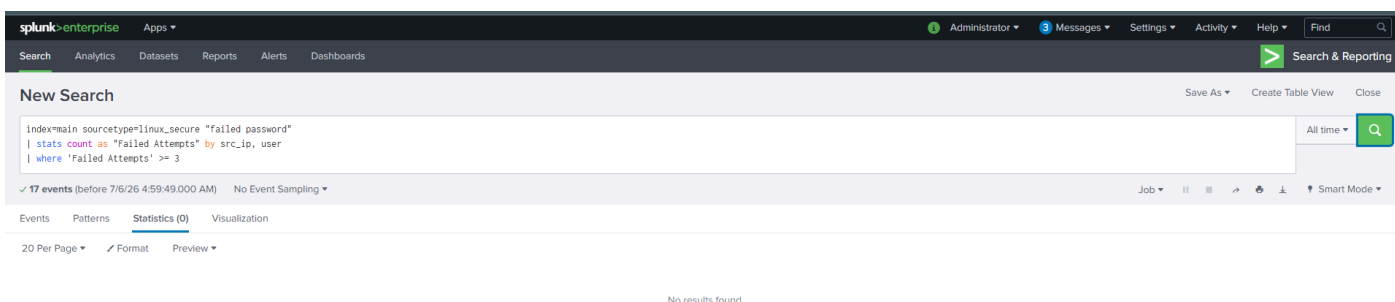
A. The Misconfiguration Pitfall (Initial Detection Failure)

An aggregate query was deployed to catch authentication brute-force patterns from the attack simulation:

```
index=main sourcetype=linux_secure "failed password"
| stats count as "Failed Attempts" by src_ip, user
| where 'Failed Attempts' >= 3
```

Screenshot Reference: [image_b423df.png]

Diagnostic Analysis: Returned "No results found." Even though 17 authentication failure events were present in the log index, the table generated zero rows because `src_ip` and `user` were not standard fields for this log format, resulting in parsing failures.



B. Inspecting Unstructured Telemetry Fields

To inspect why the query failed, a raw logging query was executed to review the unparsed strings:

```
index=main sourcetype=linux_secure "failed password"
| rename _raw as "Raw Log Event"
| table _time, "Raw Log Event"
```

Screenshot Reference: [image_b42817.png]

Diagnostic Analysis: Revealed 17 occurrences of unparsed string lines, such as:

`Failed password for vboxuser from 192.168.30.6 port 40028 ssh2`.

This verified that the text used different field naming conventions instead of the expected field objects.

Search

Analytics

Datasets

Reports

Alerts

Dashboards

Search & Reporting

New Search

Save As

Create Table View

Close

index=main sourcetype=linux_secure "failed password"
| rename _raw as "Raw Log Event"
| table _time, "Raw Log Event"

All time

✓ 17 events (before 7/6/26 5:02:15.000 AM)

No Event Sampling

Events

Patterns

Statistics (17)

Visualization

20 Per Page

Format

Preview

_time	Raw Log Event
2026-07-06 04:56:10.359	2026-07-06T04:56:10.359500+00:00 Ubuntu-SSH-Target sshd[2754]: Failed password for vboxuser from 192.168.30.6 port 40028 ssh2
2026-07-06 04:56:03.587	2026-07-06T04:56:03.587449+00:00 Ubuntu-SSH-Target sshd[2752]: message repeated 2 times: [Failed password for vboxuser from 192.168.30.6 port 52718 ssh2]
2026-07-06 04:55:53.569	2026-07-06T04:55:53.569409+00:00 Ubuntu-SSH-Target sshd[2752]: Failed password for vboxuser from 192.168.30.6 port 52718 ssh2
2026-07-06 04:55:47.731	2026-07-06T04:55:47.731852+00:00 Ubuntu-SSH-Target sshd[2750]: message repeated 2 times: [Failed password for vboxuser from 192.168.30.6 port 42512 ssh2]
2026-07-06 04:55:37.664	2026-07-06T04:55:37.664691+00:00 Ubuntu-SSH-Target sshd[2750]: Failed password for vboxuser from 192.168.30.6 port 42512 ssh2
2026-07-06 04:55:31.363	2026-07-06T04:55:31.363119+00:00 Ubuntu-SSH-Target sshd[2748]: message repeated 2 times: [Failed password for vboxuser from 192.168.30.6 port 44324 ssh2]
2026-07-06 04:55:21.776	2026-07-06T04:55:21.776218+00:00 Ubuntu-SSH-Target sshd[2748]: Failed password for vboxuser from 192.168.30.6 port 44324 ssh2
2026-07-06 04:57:31.077	2026-07-06T04:57:31.077727+00:00 Ubuntu-SSH-Target sshd[2760]: message repeated 2 times: [Failed password for vboxuser from 192.168.30.6 port 37816 ssh2]
2026-07-06 04:57:11.451	2026-07-06T04:57:11.451451+00:00 Ubuntu-SSH-Target sshd[2760]: Failed password for vboxuser from 192.168.30.6 port 37816 ssh2
2026-07-06 04:57:04.179	2026-07-06T04:57:04.179806+00:00 Ubuntu-SSH-Target sshd[2758]: message repeated 2 times: [Failed password for vboxuser from 192.168.30.6 port 53524 ssh2]
2026-07-06 04:56:51.936	2026-07-06T04:56:51.936522+00:00 Ubuntu-SSH-Target sshd[2758]: Failed password for vboxuser from 192.168.30.6 port 53524 ssh2
2026-07-06 04:56:45.576	2026-07-06T04:56:45.576210+00:00 Ubuntu-SSH-Target sshd[2756]: message repeated 2 times: [Failed password for vboxuser from 192.168.30.6 port 54528 ssh2]
2026-07-06 04:56:28.243	2026-07-06T04:56:28.243741+00:00 Ubuntu-SSH-Target sshd[2756]: Failed password for vboxuser from 192.168.30.6 port 54528 ssh2
2026-07-06 04:56:20.554	2026-07-06T04:56:20.554454+00:00 Ubuntu-SSH-Target sshd[2754]: message repeated 2 times: [Failed password for vboxuser from 192.168.30.6 port 40028 ssh2]
2026-07-06 04:57:48.480	2026-07-06T04:57:48.480526+00:00 Ubuntu-SSH-Target sshd[2762]: message repeated 2 times: [Failed password for vboxuser from 192.168.30.6 port 43802 ssh2]
2026-07-06 04:57:37.714	2026-07-06T04:57:37.714964+00:00 Ubuntu-SSH-Target sshd[2762]: Failed password for vboxuser from 192.168.30.6 port 43802 ssh2
2026-07-06 04:57:54.223	2026-07-06T04:57:54.223052+00:00 Ubuntu-SSH-Target sshd[2764]: Failed password for vboxuser from 192.168.30.6 port 47192 ssh2

C. Corrective Engineering: Regular Expression Parsing Rule

To resolve this, inline Regular Expression parsing rules were introduced to dynamically isolate and extract these text patterns:

```
index=main sourcetype=linux_secure "failed password"
| rex "Failed password for (?<target_user>\S+) from (?<attacker_ip>\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})"
| stats count as "Failed Attempts" by attacker_ip, target_user
| where 'Failed Attempts' ≥ 3
```

Diagnostic Analysis: Resolves the parsing failure. The regex accurately isolates the unstructured text fields, grouping the data to display 1 row showing `attacker_ip: 192.168.30.6` , `target_user: vboxuser` , and a count of 17 `Failed Attempts` .

Competency Domain	Lab Implementation Mechanism	Operational Impact
-------------------	------------------------------	--------------------

Infrastructure Isolation	Secondary virtualization switch fabric utilizing distinct host-only subnets (192.168.30.0/24).	Ensures zero-leakage testing environments, protecting corporate production subnets from live security validation drills.
Host Hardening & Logging	System-wide rollout of Microsoft Sysmon v15.21 utilizing specialized XML configuration schemas.	Establishes deep kernel-level telemetry collection (process creation, network linkages) necessary for reliable forensic post-mortems.
Defensive Parser Engineering	Transitioning raw unstructured log views into structured statistical matrices via custom regular expressions.	Mitigates the risk of false negatives caused by field-mismatching, ensuring SOC analytics scale accurately across diverse multi-vendor banking platforms.

6.3 Final Engineering Status

[STATUS REPORT]: this lab demonstrates foundational SIEM ingestion and detection engineering skills relevant to regulated environments

Complete Summary of entire Project/Lab : -

In this engineering project, I successfully designed and deployed a fully isolated layer-2 host-only network topology (192.168.30.0/24) to simulate a banking-grade SOC environment. I provisioned and proactively hardened my target assets by deploying Microsoft Sysmon v15.21 onto a Windows 10 endpoint, configuring a target Ubuntu server, and updating an Arch Linux node with an offensive testing suite. When network barricades threatened connectivity, I systematically diagnosed and resolved the layer-2 issues by flushing stale ARP neighbor tables and executing PowerShell firewall overrides to permit critical diagnostic traffic. Throughout this deployment, troubleshooting lifecycle, and parsing phases, I integrated **AI Gemini** as a core technical collaborator to help analyze log structures and optimize detection logic. While certain intermediate diagnostic commands, terminal outputs, and granular steps were omitted from this documentation due to an untracked command history during active execution, the critical milestones were successfully preserved. With a stable communication fabric established, I launched automated, high-velocity credential-stuffing simulations using THC-Hydra against both SSH and RDP daemons to generate predictable event logs, such as Event ID 4625. Finally, I forwarded this multi-node telemetry to a centralized Splunk SIEM pipeline where, upon uncovering an unparsed telemetry pitfall, I engineered custom inline regular expression (\$rex\$) parsing rules to extract vital threat fields like `attacker_ip` and `target_user`.